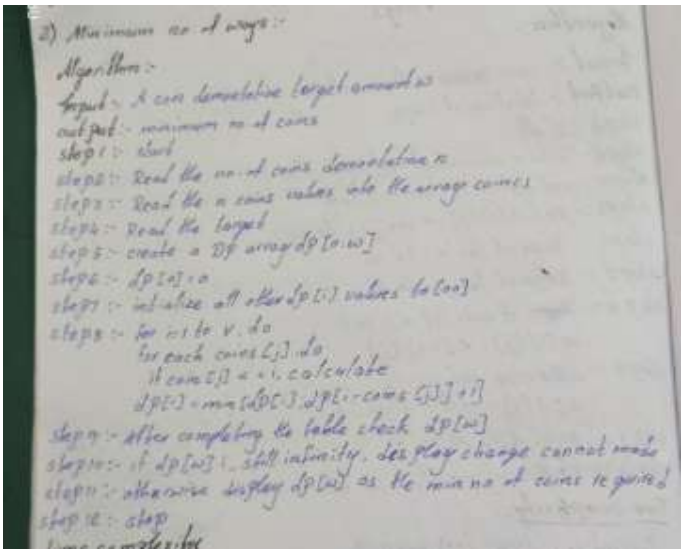
PRACTICAL: 07

AIM:- Implementation of making a change problem using dynamic programming

Algorithms:-



Program/code:-

```
#include <iostream>
using namespace std;
int main()
{
int n, amount;
cout << "Enter number of coins: ";
cin >> n;
```

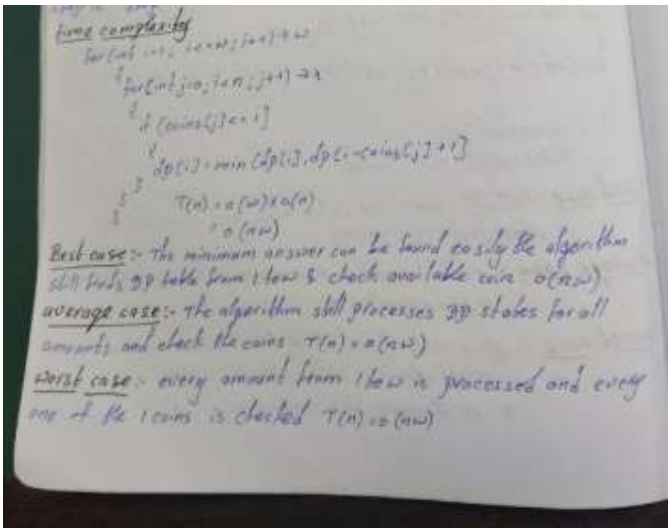
```
int coin[n];
cout << "Enter coin denominations: ";
for (int i = 0; i < n; i++)
{
cin >> coin[i];
}
cout << "Enter amount: ";
cin >> amount;
int dp[amount + 1];
dp[0] = 0;
for (int i = 1; i <= amount; i++)
{
dp[i] = 32767;
for (int j = 0; j < n; j++)
{
if (coin[j] <= i)
{
dp[i] = min(dp[i], dp[i - coin[j]] + 1);
}
}
}
cout << "Minimum number of coins required = "
<< dp[amount] << endl;
return 0;
}
```

Output:-

```
Enter number of coins: 4
Enter coin denominations: 5
4
2
3
Enter amount: 7
Minimum number of coins required = 2


...Program finished with exit code 0
Press ENTER to exit console.
```

Time complexity:-



Conclusion:-

Dynamic programming (DP) is the most effective approach for solving the Change-Making Problem because it guarantees a globally optimal solution. While a greedy algorithm is faster, it can fail to find the minimum number of coins for arbitrary coin denominations. By breaking the problem down into overlapping subproblems and storing the results in a table (memoization or tabulation), DP avoids redundant computations and handles any coin set efficiently.